AURA-UDP implementation progress report

Review 2 milestone • Approximately 70% complete
Deadline-aware reliable transport for critical campus IoT messages

This report explains the project for a reader who has not seen the code. It records what is implemented and verified, what the NS2 experiments found, what Cisco Packet Tracer demonstrates, and what remains for the final review. The 70% figure is a planning estimate of overall project readiness, not a measured software metric.

The project in one minute

A smart campus sends routine readings and urgent alerts to a monitoring room. A lost or late routine reading may be tolerable; a smoke alert is useful only if it arrives before its deadline. AURA-UDP asks whether a sender can improve timely alert delivery while controlling the cost of retransmissions.

Two tools show different parts of that question. NS2 runs the actual transport experiment and produces timing, loss, and retry measurements. Cisco Packet Tracer is an editable campus network model that shows routers, HTTP connectivity, an IoT gateway, and registered sensor devices. The tools do not exchange live packets.

What the 70% milestone covers

Workstream	Implemented and evidenced	Status
Transport in NS2	Baseline, Selective Repeat, and AURA sender/receiver behavior; ACKs, deadlines, retry and congestion-window decisions.	Built
Evaluation	Three scenarios, paired seed sweeps, policy ablations, fresh-seed checks, metrics, charts, and a local dashboard.	Built
Campus in Packet Tracer	Two routers, monitoring server, native temperature and smoke devices, gateway registration, routed HTTP check.	Built
Final research claim and demo	Wider workloads, stronger fairness/congestion tests, improved retry estimate, and final presentation evidence.	Remaining

How the pieces fit

Campus alert idea → NS2 sender and lossy link → event logs and metrics → comparison charts and dashboard

Campus layout idea → Packet Tracer routers, server, gateway, and sensors → connectivity and registration screenshots

What happens inside NS2

The script ns2/run.tcl builds a sender, router, monitoring receiver, and competing CBR sender. Its 128 Kb/s bottleneck has a short queue and configurable packet loss. Each run creates 30 messages; every fourth is critical and has an absolute deadline. Data and ACK paths have separate loss streams.

Mode	Sender behavior	Purpose
Baseline UDP	Sends a message once; no ACK or retry.	Reference for low overhead.
Fixed Selective Repeat	Uses a four-message window, ACKs and individual retries with a fixed timer.	Reference reliable method.
AURA	Orders urgent alerts by deadline, gives routine traffic regular turns, adapts timer and small window, and checks whether a retry can still arrive in time.	Tests the deadline-aware policy.

The decision being tested

For a critical message, the sender compares estimated forward delivery time with time remaining until its absolute deadline. If the estimated trip is longer, AURA abandons that retransmission. The first send is still attempted while the alert is live. This can save traffic, but a rough estimate may reject a retry that would have succeeded.

Retry when estimated delivery time ≤ deadline − current time

What is measured

- Deadline delivery ratio (DDR): critical messages first delivered on time divided by all generated critical messages, including lost and abandoned ones.
- Packet delivery ratio (PDR): unique messages delivered eventually divided by all generated messages. A late alert can count in PDR but not DDR.
- Also recorded: delay for delivered messages, extra sends per generated message, routine delivery, and received background traffic.

NS2 writes events.csv, control.csv, background.csv, a packet trace, and a NAM trace. Python analysis deduplicates message IDs and produces summary CSVs, plots, and the local dashboard. The dashboard reads saved results; it does not run NS2.

Experiment design

The development set uses 30 seed indices across three conditions and three transport modes (270 runs), plus four AURA variants (360 runs). A fresh 30-seed set repeats both sweeps. Total recorded evaluation: 1,260 NS2 runs. Within a scenario, modes use the same topology, generation schedule, and seed index; changed retry behavior means packet-by-packet loss draws are not identical.

NS2 evidence and measured findings

The figure below is generated from the checked-in NS2 run summaries. It compares delivery and retry behavior across moderate loss, tight deadlines, and a higher message rate. It is experimental output, not Packet Tracer traffic.

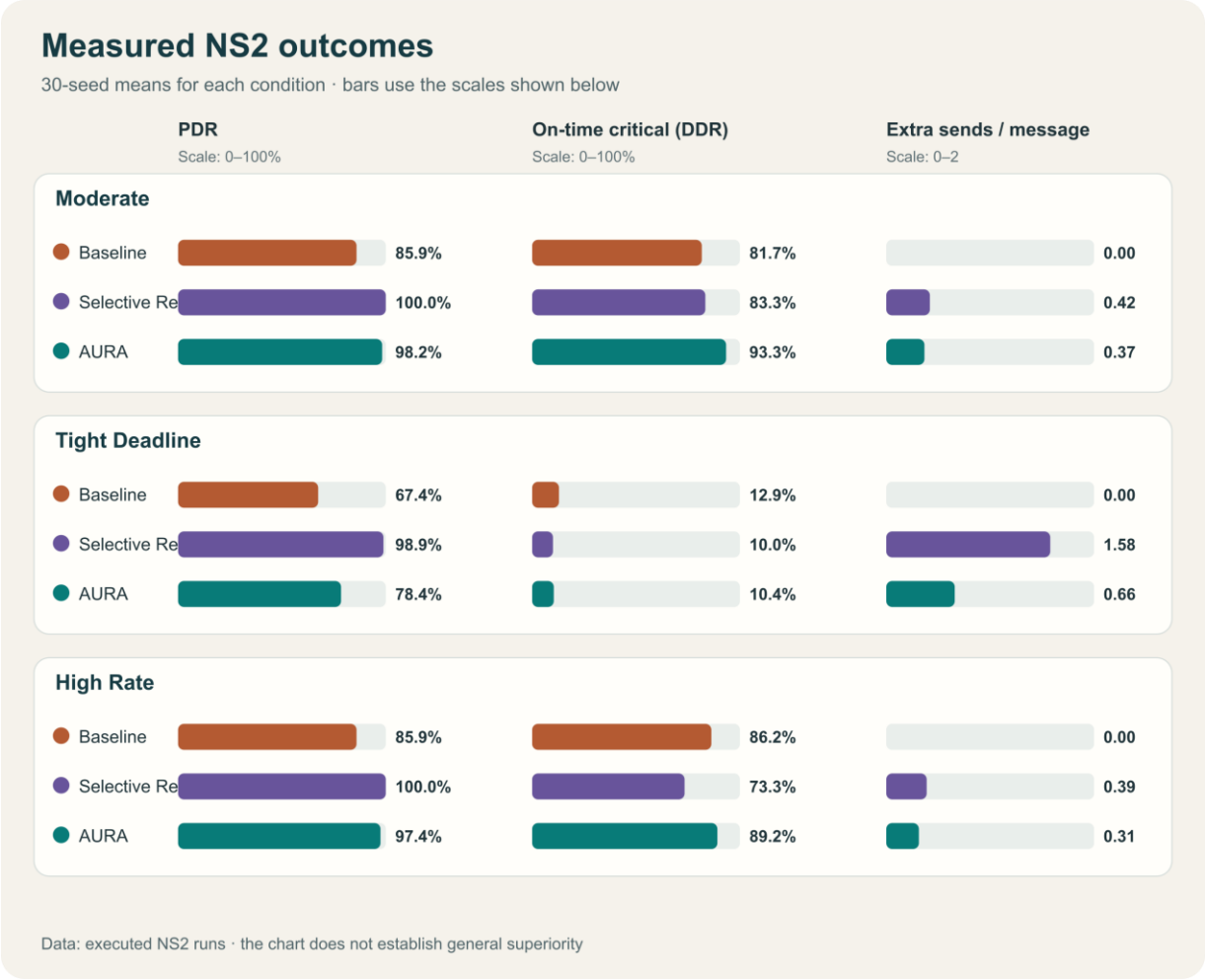


Figure 1 NS2 30-seed comparison from experiments/plots/multiseed.svg

Scenario	Baseline DDR	Fixed SR DDR	AURA DDR
Moderate loss	81.7%	83.3%	93.3%
Tight deadline	12.9%	10.0%	10.4%
High message rate	86.3%	73.3%	89.2%

AURA delivered more critical messages on time than fixed SR in the moderate and high-rate conditions. The tight-deadline condition remained difficult for every mode, and baseline exceeded AURA there. The feasibility check reduced high-rate retry cost in the development set, but did not show a DDR gain; on fresh seeds, full AURA and the version without this check both reached 90.4% DDR. These are descriptive results for three short synthetic workloads.

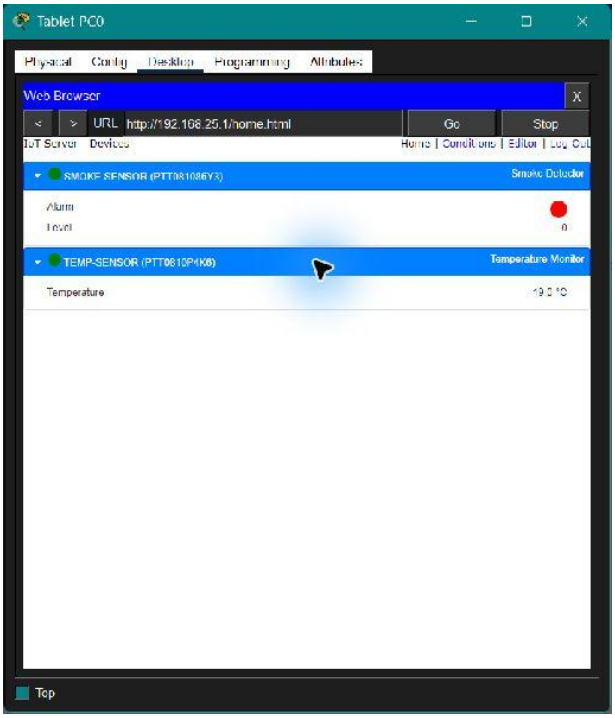
Cisco Packet Tracer implementation

The editable packet_tracer/topology/AURA-UDP-campus.pkt models the campus side of the project. PC0 on the sensor LAN reaches Server0 at 10.10.30.10 through Router0 and Router1. Separately, a native Temperature Monitor and Smoke Detector register with HomeGateway0; Tablet PC0 displays them at 192.168.25.1.

The original captures below are kept at a modest size because Packet Tracer exported them at only 513 pixels wide. Their captions state the tested observation so the small text is not the only evidence.



PC0 browser: routed HTTP page served by Server0.



Tablet gateway view: smoke and temperature devices registered.

Figures 2 and 3 Original Cisco Packet Tracer screenshots

What these checks establish

- The saved PC0 browser loaded the custom server page over the two-router path.
- The gateway device list showed both native IoT devices and their displayed fields.

Packet Tracer does not execute AURA or feed the NS2 simulation. A triggered smoke alarm transition has not been verified. The gateway and wireless settings are a lab demonstration, not a production security design.

Verification and remaining work

The repository records 19 passing Python and NS2 checks, repeatable sweep outputs, and a GitHub Actions workflow that runs tests and small NS2 smoke sweeps. The saved Packet Tracer checks cover routed HTTP, pings, and native IoT registration. Versioned Git commits and releases provide rollback points.

Remaining 30% for the final review

- Test longer and more varied workloads, more competing traffic, and a wider range of deadlines and loss conditions.
- Improve the forward-time estimate with queueing uncertainty and evaluate whether the deadline decision improves DDR without starving routine traffic.
- Strengthen congestion and fairness evaluation; the current bounded window is a lab heuristic, not an Internet-grade controller.
- Capture a verified smoke-event transition in Packet Tracer and prepare a concise live demonstration and final report.

How to reproduce the evidence

Open packet_tracer/topology/AURA-UDP-campus.pkt in Packet Tracer 9.0. On PC0, browse to http://10.10.30.10. On Tablet PC0, browse to http://192.168.25.1 and inspect the gateway device list.

In Ubuntu or WSL with NS2 2.35 installed, run python3 scripts/multiseed.py and python3 scripts/ablation.py from the repository root. Open experiments/plots/multiseed.svg and docs/RESEARCH_EVALUATION.md for the measured comparison. To inspect the local dashboard, start python3 -m http.server 8000 --bind 127.0.0.1 from the repository root, then open http://localhost:8000/python/dashboard/.

Source files in the repository

File or folder	What it contains
ns2/run.tcl	Topology, transport behavior, deadlines, ACKs, and logs.
scripts/ and python/analysis/	Experiment runners, metrics, and plots.
experiments/results/	Per-run and summary data for development and fresh seeds.
packet_tracer/	Editable campus topology, web page, build guide, and captures.
docs/PROTOCOL.md and docs/VALIDATION.md	Decision rules and verification record.

Review 2 conclusion: the core simulation, comparison pipeline, campus model, and presentation evidence are implemented. The final phase should focus on broader validation and a clearer demonstration of when the deadline-aware decision helps or fails.